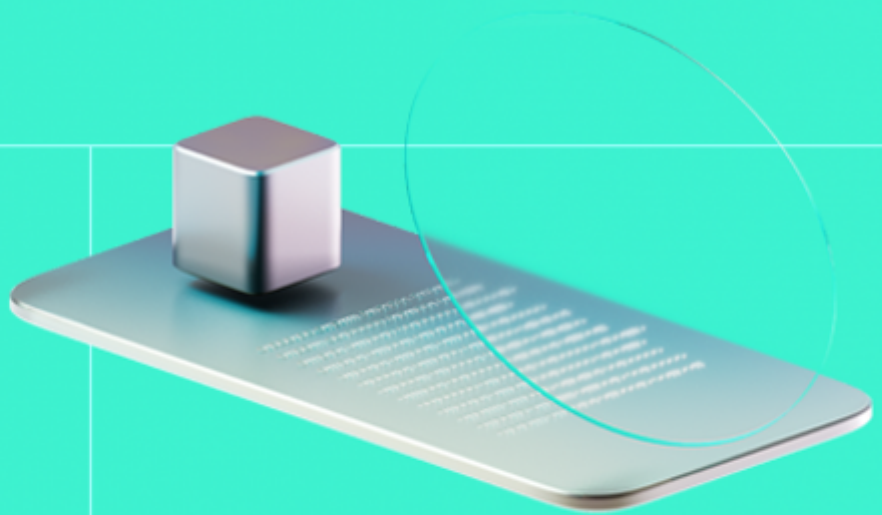




Smart Contract Code Review And Security Analysis Report

Customer: EverValue Coin

Date: 12/07/2024



We express our gratitude to the EverValue Coin team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

EverValue Coin (EVA) is an ERC20 token designed to work on the Arbitrum network, offering a decentralized finance (DeFi) ecosystem centered around token burns and exchanges. EVA is burnable and interacts with various smart contracts, allowing holders to increase their value by burning tokens for a share of wBTC reserves, thus creating a dynamic share system.

Platform: Arbitrum(EVM)

Language: Solidity

Tags: ERC20, wBTC, Token Sale,

Timeline: 22/05/2024 - 23/05/2024

Methodology: https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/devervalue/evervaluecoin
Commit	96585f2

Audit Summary

10/10

8/10

100%

10/10

Security score

Code quality score

Test coverage

Documentation quality score

Total 9.6/10

The system users should acknowledge all the risks summed up in the risks section of the report

1

1

0

0

Total Findings

Resolved

Accepted

Mitigated

Findings by severity

Critical	0
High	0
Medium	1
Low	0

Vulnerability	Status
F-2024-4200 - Sales made with low amounts can be made for free due to lack of control	Fixed

This report may contain confidential information about IT systems and the intellectual property of the Customer, as well as information about potential vulnerabilities and methods of their exploitation.

The report can be disclosed publicly after prior consent by another Party. Any subsequent publication of this report shall be without mandatory consent.

Document

Name	Smart Contract Code Review and Security Analysis Report for EverValue Coin
Audited By	Eren Gönen
Approved By	Ataberk Yavuzer
Changelog	28/05/2024 - Preliminary Report 03/07/2024 - Final Report

Table of Contents

System Overview	6
Privileged Roles	6
Executive Summary	7
Documentation Quality	7
Code Quality	7
Test Coverage	7
Security Score	7
Summary	7
Risks	8
Findings	9
Finding Details	9
Observation Details	11
Disclaimers	18
Appendix 1. Severity Definitions	19
Appendix 2. Scope	20

System Overview

EverValue Coin is an ERC20 token with a burnable feature that interacts with other contracts like **EVAMarket** and **EVABurnVault**. These contracts allow users to buy, sell, or burn EVA tokens and receive market tokens or wBTC in return. The following contracts are involved:

EverValueCoin — A burnable ERC20 token with a limited supply, designed to work on the Arbitrum network. This contract manages the minting and burning of EVA tokens, with a capped total supply of 21,000,000 tokens.

It has the following attributes:

- Name: EverValueCoin
- Symbol: EVA
- Decimals: 18
- Total supply: 21 m tokens.

EVABurnVault — A vault that allows users to burn EVA tokens in exchange for backing wBTC tokens. The contract facilitates the burning of EVA tokens and ensures fair distribution of wBTC tokens.

EVAMarket — A centralized market for buying and selling EVA tokens at administratively defined rates. The contract allows users to buy or sell EVA tokens using a defined market token, with adjustable rates and fees.

Privileged roles

- The owner of the **EVAMarket** contract can set the transaction fee for selling EVA tokens and withdraw all tokens from the contract.

Executive Summary

This report presents an in-depth analysis and scoring of the customer's smart contract project. Detailed scoring criteria can be referenced in the [scoring methodology](#).

Documentation quality

The total Documentation quality score is **10** out of **10**.

- Functional requirements are provided
- Technical description is provided.

Code quality

The total Code quality score is **8** out of **10**.

- The development environment is configured.
- Some best practices are missed.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Everything covered with tests.

Security score

Upon auditing, the code was found to contain **0** critical, **0** high, **1** medium, and **0** low severity issues. Out of these, **1** issues have been addressed and resolved, leading to a security score of **10** out of **10**.

All identified issues are detailed in the “Findings” section of this report.

Summary

The comprehensive audit of the customer's smart contract yields an overall score of **9.6**. This score reflects the combined evaluation of documentation, code quality, test coverage, and security aspects of the project.

Risks

- **Solidity Version Compatibility:** The Solidity version 0.8.20 employs the recently introduced PUSH0 opcode in the Shanghai EVM. This opcode might not be universally supported across all blockchain networks and Layer 2 solutions. Thus, as a result, it might be not possible to deploy solution with version 0.8.20 $\geq$ on some blockchains.
- **Centralized Minting to a Single Address:** The project concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- **Absence of Pausable Feature for Unexpected Events:** Without the pausable feature, the contract cannot be immediately paused in the event of unexpected occurrences.
- **Insufficient Multi-signature Controls for Critical Functions:** The lack of multi-signature requirements for key operations centralizes decision-making power, increasing vulnerability to single points of failure or malicious insider actions, potentially leading to unauthorized transactions or configuration changes.

Findings

Finding Details

[F-2024-4200](#) - Sales made with low amounts can be made for free due to lack of control - Medium

Description: During the remediation commit, it was observed that there is a possibility to complete sell operations for free due to precision loss. It has been determined that in the scenario where there is a decimal difference between the Market token and the EverValue token and small amounts of EverValue token are sold, the sale transaction takes place and there is no token transfer in return for the sale.

```
uint256 marketTokenToTransfer = (((normalizedAmount *
marketTokenPer100Eva) / 100) * (1000 - fee)) / 1000;
require(
    marketTokenToTransfer <= marketToken.balanceOf(address(this)),
    "Market doesn't have enough balance"
);

eva.safeTransferFrom(msg.sender, this.owner(), amount);
marketToken.safeTransfer(msg.sender, marketTokenToTransfer);
```

Additional control should be provided to prevent free sales in the protocol. It is important to add such controls to prevent potential losses.

Status: Fixed

Classification

Impact: 4/5
Likelihood: 2/5
Exploitability: Independent
Complexity: Simple
Severity: Medium

Recommendations

Remediation:

It is recommended to add a sanity check which controls if the `marketTokenToTransfer` variable is higher than zero.

Resolution:

The **EverValue** team implemented the recommended check to eliminate this finding in given commit (4631b7).

Observation Details

[F-2024-3270](#) - Missing Events - Info

Description: Events for critical state changes should be emitted for tracking actions off-chain.

It was observed that events are missing in the following functions:

```
setRate()  
setFee()
```

Events are crucial for tracking changes on the blockchain, especially for actions that alter significant contract states or permissions. The absence of events in these functions means that external entities, such as user interfaces or off-chain monitoring systems, cannot effectively track these important changes.

Assets:

- EVAMarket.sol [<https://github.com/devervalue/evervaluecoin>]

Status: Accepted

Recommendations

Remediation: Consider implementing and emitting events for the necessary functions.

Resolution: This observation is acknowledged by the **EverValue** team.

[F-2024-3271](#) - Functions that Can Be Declared External - Info

Description: There functions which are set to public visibility, despite it never being called internally within the contract.

EverValueCoin contract:

- burn function
- burnFrom function

EVABurnVault contract:

- backingWithdraw function

EVAMarket contract:

- buy function
- sell function
- setRate function
- setFee function
- withdrawAll function

Using public visibility is more costly in terms of Gas compared to external, leading to unnecessary Gas usage.

Assets:

- EVABurnVault.sol [<https://github.com/devvalue/evervaluecoin>]
- EverValueCoin.sol [<https://github.com/devvalue/evervaluecoin>]
- EVAMarket.sol [<https://github.com/devvalue/evervaluecoin>]

Status:

Accepted

Recommendations

Remediation: Consider changing the visibility of the mentioned functions from public to external.

Resolution: The finding was acknowledged by the **EverValue** team.

[F-2024-3272](#) - Redundant imports - Info

Description: The contracts imports **IERC20** from OpenZeppelin:

```
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
```

Since the import of **./EverValueCoin.sol** already includes an **IERC20** interface, the direct import of **IERC20** through OpenZeppelin becomes redundant.

Assets:

- EVABurnVault.sol [<https://github.com/devvalue/evervaluecoin>]
- EVAMarket.sol [<https://github.com/devvalue/evervaluecoin>]

Status:

Accepted

Recommendations

Remediation: Remove the redundant imports.

Resolution: The finding was acknowledged by the **EverValue** team.

[F-2024-3273](#) - State Variables Only Set in The Constructor Should Be Declared `immutable` - Info

Description: Compared to regular state variables, the gas costs of constant and immutable variables are much lower. Immutable variables are evaluated once at construction time and their value is copied to all the places in the code where they are accessed.

```
20: address public wbtcAddress;
```

These variables can be declared immutable This will lower the Gas taxes.

Assets:

- EVABurnVault.sol [<https://github.com/devervalue/evervaluecoin>]

Status: Accepted

Recommendations

Remediation: Consider marking state variables as an immutable that never changes on the contract.

Resolution: The finding was acknowledged by the **EverValue** team.

[F-2024-3274](#) - Redundant `\_msgSender()` , Meta-Transactions Not Implemented - Info

Description: The `_msgSender()` function is needed to handle the meta transactions, in the OpenZeppelin library, it is used to support the development of the libraries which might be used with the contracts with the specified `TrustedForwarder` or to be used in such contracts directly.

However, the current implementation is not a library and does not rewrite the `_msgSender()` function to support meta transactions. This leads to the redundancy, because the system do not utilize `_msgSender()` features and used only as a substitute for `msg.sender`.

Assets:

- `EverValueCoin.sol` [<https://github.com/devervalue/evervaluecoin>]

Status: Accepted

Recommendations

Remediation: If `_msgSender()` is not going to be used in current implementation to support the meta transactions it is recommended to use `msg.sender` to prevent unforeseen issues and misinterpretations that may occur after the future updates.

Resolution: The finding was acknowledged by the **EverValue** team.

[F-2024-3279](#) - Lack of Upper Bound on `marketTokenPer100Eva` Parameter - Info

Description: The `setRate()` function do not impose a limit on the `marketTokenPer100Eva` parameter, which represents the price for 100 EVA in market tokens. This omission can result in miscalculation of `evaToUser` or `marketTokenToTransfer`, potentially causing incorrect fund distribution.

The buy function calculates `evaToUser` as follows:

```
uint256 evaToUser = (amount * 100) / marketTokenPer100Eva;
```

The sell function calculates `marketTokenToTransfer` as follows:

```
uint256 marketTokenToTransfer = (((amount * marketTokenPer100Eva) / 100) * (1000
```

Without a limit, if the `marketTokenPer100Eva` is set too high, the calculation of `evaToUser` or `marketTokenToTransfer` will yield unreasonable values.

Assets:

- EVAMarket.sol [<https://github.com/devervalue/evervaluecoin>]

Status:

Accepted

Recommendations

Remediation: Implement a check to ensure that the `marketTokenPer100Eva` parameter does not exceed a reasonable maximum value.

Resolution: The finding was acknowledged by the **EverValue** team.

[F-2024-3282](#) - Inadequate Management of Transaction Fees in Token Sale Function - Info

Description: The `sell()` function deducts a transaction fee from the tokens transferred to users during the selling process of EVA tokens. This fee deduction is computed as part of the `marketTokenToTransfer` calculation:

```
function sell(uint256 amount) public {  
    ...  
    uint256 marketTokenToTransfer = (((amount * marketTokenPer100Btt) /  
        100) * (1000 - fee)) / 1000;  
    require(  
        marketTokenToTransfer <= marketToken.balanceOf(address(this)),  
        "Market doesn't have enough balance"  
    );  
  
    btt.safeTransferFrom(msg.sender, this.owner(), amount);  
    marketToken.safeTransfer(msg.sender, marketTokenToTransfer);  
  
    emit userSold(marketTokenToTransfer, amount);  
}
```

The contract does not implement any mechanism to manage or store the deducted fees. This lack of fee management could lead to discrepancies in the handling of funds, where the deducted fees remain unaccounted within the overall contract's balance, potentially causing inconsistencies.

Assets:

- EVAMarket.sol [<https://github.com/devvalue/evervaluecoin>]

Status:

Accepted

Recommendations

Remediation: Implement a fee management system within the contract that collects and stores transaction fees in a designated wallet or internal contract variable.

Resolution: The finding was acknowledged by the **EverValue** team.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Severity Definitions

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution, do not affect security score but can affect code quality score.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details

Repository	https://github.com/devervalue/evervaluecoin
Commit	96585f24084b3be7936a1288c20e439e78d0c251
Remediation Commit	4631b7b8241f5c08313a2222d16187b847fa90df
Whitepaper	N/A
Requirements	https://github.com/devervalue/evervaluecoin/blob/main/API.md
Technical Requirements	https://github.com/devervalue/evervaluecoin/blob/main/README.md

Contracts in Scope

./contracts/EverValueCoin.sol

./contracts/EVABurnVault.sol

./contracts/EVAMarket.sol

Deployed Contracts Arbitrum

addrEva	0x45d9831d8751b2325f3dbf48db748723726e1c8c
addrBurnVault	0xA89d65deF0A001947d8D5fDda93F9C4f8453902e
addrMarket	0x53314ba4c6015466b881d614F3840CF0d038e94C